

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Concept Mapping

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	35% of CIA	Total Marks:	100
CO1 Statement:	Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.		
Student Name:		Reg. No.:	
Section / Batch:		Date:	10/07/2026

Code of Conduct

I _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 Concept mapping questions. Total: 100 Marks.
- When a student answer using a concept map, in evaluation the answer should be with following four requirements: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Linked data structure	Basic data structure	1	20
Primitive and Non-Primitive Data Structure - Linear and Non-Linear Data Structure	Classifications of Linear data structure	2	20
Search Data Structure	Linear Search and Binary Search	3	20
Recursion, Performance analysis	Recursion	4	20
Tree Data Structures	Classifications of Tree data structure	5	20
GRAND TOTAL:		100 Marks	

Annexure A – Concept Mapping

1. A student attendance system records daily presence data. Construct a concept map linking Array → Fixed Size → Sequential Access → Linked List → Dynamic Allocation → Flexibility. Explain how each structure works and justify which is more suitable for dynamic data updates. (20 Marks)
2. A weather monitoring system collects temperature, humidity, and wind data continuously. Construct a concept map linking Primitive Data Types → Non-Primitive Structures → Linear Structures → Non-Linear Structures → Applications. Include examples and explain how each structure supports continuous data processing. (20 Marks)
3. A digital archive stores millions of documents. Construct a concept map linking Searching → Linear Search → Binary Search → Sorted Data → Time Complexity → Performance. Analyze the impact of data organization on search efficiency and justify the best technique. (20 Marks)
4. A program processes nested expressions using recursion and may encounter deep nesting. Construct a concept map linking Recursion → Call Stack → Stack Overflow → Error Handling → Efficiency. Analyze potential issues and justify strategies to handle them. (20 Marks)
5. A company maintains hierarchical employee records with multiple levels of management. Construct a concept map connecting Tree → Nodes → Levels → Traversal (DFS/BFS) → Searching → Efficiency. Analyze how traversal techniques affect performance and justify the most suitable approach for retrieving employee data. (20 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Student Attendance System: Arrays vs Linked Lists



Student Attendance System

- Core purpose
 - Student attendance records store daily presence, absence, and related status for each student
 - Data structure choice affects update speed, access speed, memory use, and scalability
 - System goal: balance efficient tracking with efficient record modification
- Data structure comparison focus
 - Array: stronger for fast writes and stable data
 - Linked List: stronger for frequent changes and flexible record growth
 - Decision factor: whether a student's record is mostly fixed or frequently updated

Conclusion

- Final recommendation
 - Linked List is more suitable for dynamic attendance systems
 - Linked List supports dynamic memory allocation
 - Linked List provides flexible size
 - Linked List enables efficient insertion and deletion of student records without shifting existing data

Comparison

- Access and performance
 - Array: faster random access
 - Linked List: better for frequent insertions and deletions
 - Array: efficient when direct indexing is needed
 - Linked List: efficient when records change often
- Scalability and use case
 - Array: suitable for fixed number of students
 - Linked List: suitable for changing student lists
 - Array: ideal for stable attendance datasets
 - Linked List: ideal for dynamic attendance systems
- Trade-offs
 - Array trade-off: higher access speed but less flexible updates
 - Linked List trade-off: flexible updates but slower direct access
 - Array trade-off: possible memory wastage with unused capacity
 - Linked List trade-off: extra memory overhead for pointers

Array

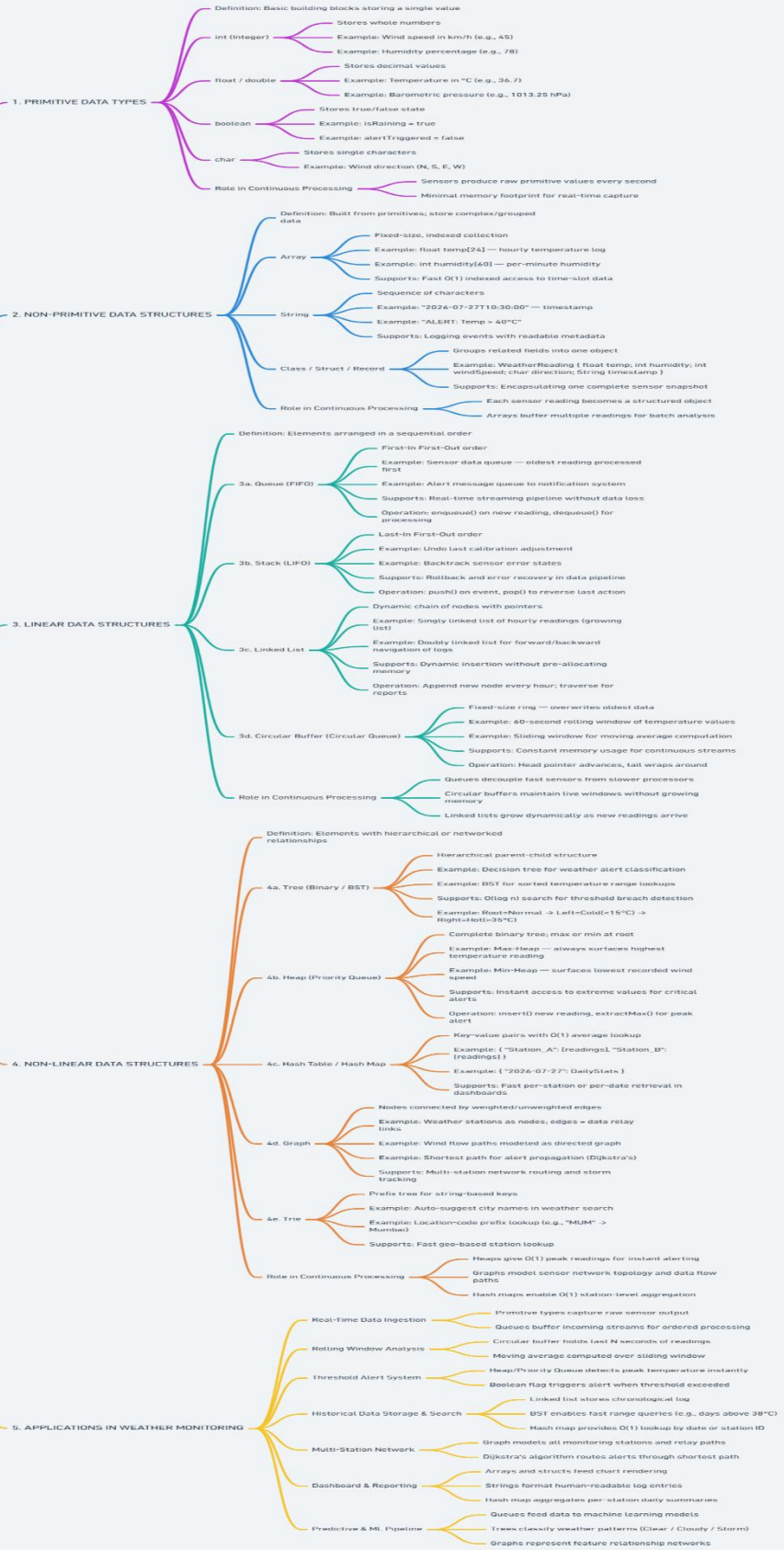
- Structure and storage
 - Array: fixed-size storage for attendance records
 - Array: contiguous memory allocation
 - Array: sequential access supports easy traversal
 - Fast index-based access for direct student record lookup
 - Sequential access provides efficient update of daily attendance
 - Easy traversal helps scan full class records quickly
- Operational characteristics
 - Insertion and deletion require shifting elements
 - Shifting elements can slow down record changes
 - Memory wastage occurs if extra space is allocated
 - Best suited for static attendance records
 - Stable for fixed number of students
 - Fast random access supports quick lookup by position
- Array relationships
 - Array → fixed size
 - Fixed Size → provides → Sequential Access
 - Sequential Access → improves → Efficient Retrieval of Daily Attendance
 - Array → supports → Fast Index-Based Access
 - Array → requires → Element Shifting for Insertion and Deletion
 - Array → results in → Memory Wastage if Extra Space is Allocated
 - Array → best suited for → Static Attendance Records

Linked List

- Structure and storage
 - Nodes connected by pointers
 - Records also supports growth and shrinkage
 - No need for contiguous memory
 - Sequential traversal for record reading
 - Nodes store links to next record
 - Pointer-based structure enables flexible record organization
- Operational characteristics
 - Easy insertion and deletion
 - No shifting of existing data required
 - Additional memory needed for pointers
 - Best suited for frequently updated attendance records
 - Suitable for changing student lists
 - Efficient when new records added or removed often
- Array relationships
 - Linked List → has → Dynamic Allocation
 - Dynamic Allocation → provides → Flexibility
 - Flexibility → supports → Efficient Dynamic Data Updates
 - Linked List → allows → Easy Insertion and Deletion
 - Linked List → uses → Nodes Connected by Pointers
 - Linked List → requires → Additional Memory for Pointers
 - Linked List → best suited for → Frequently Updated Attendance Records

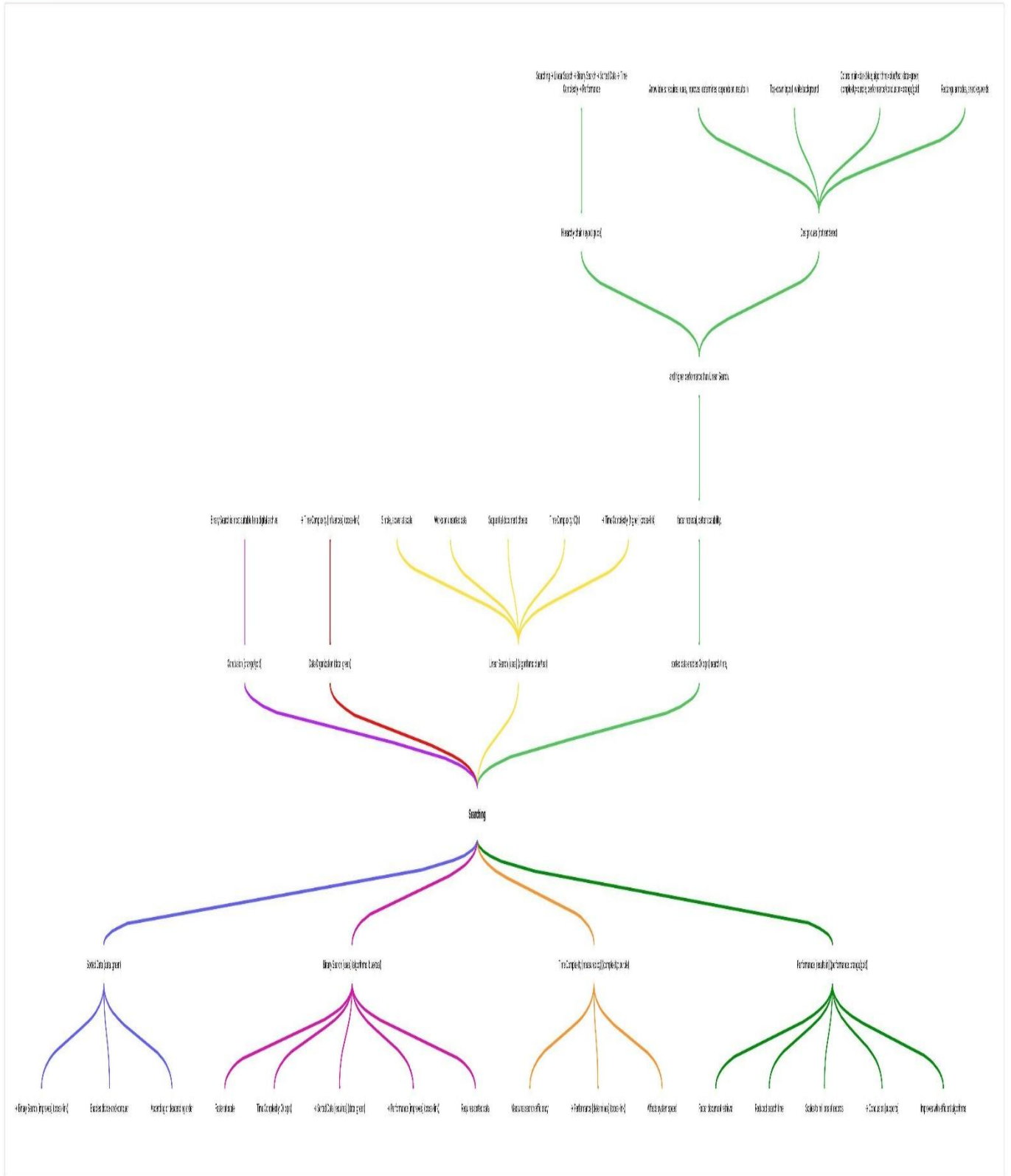
2.

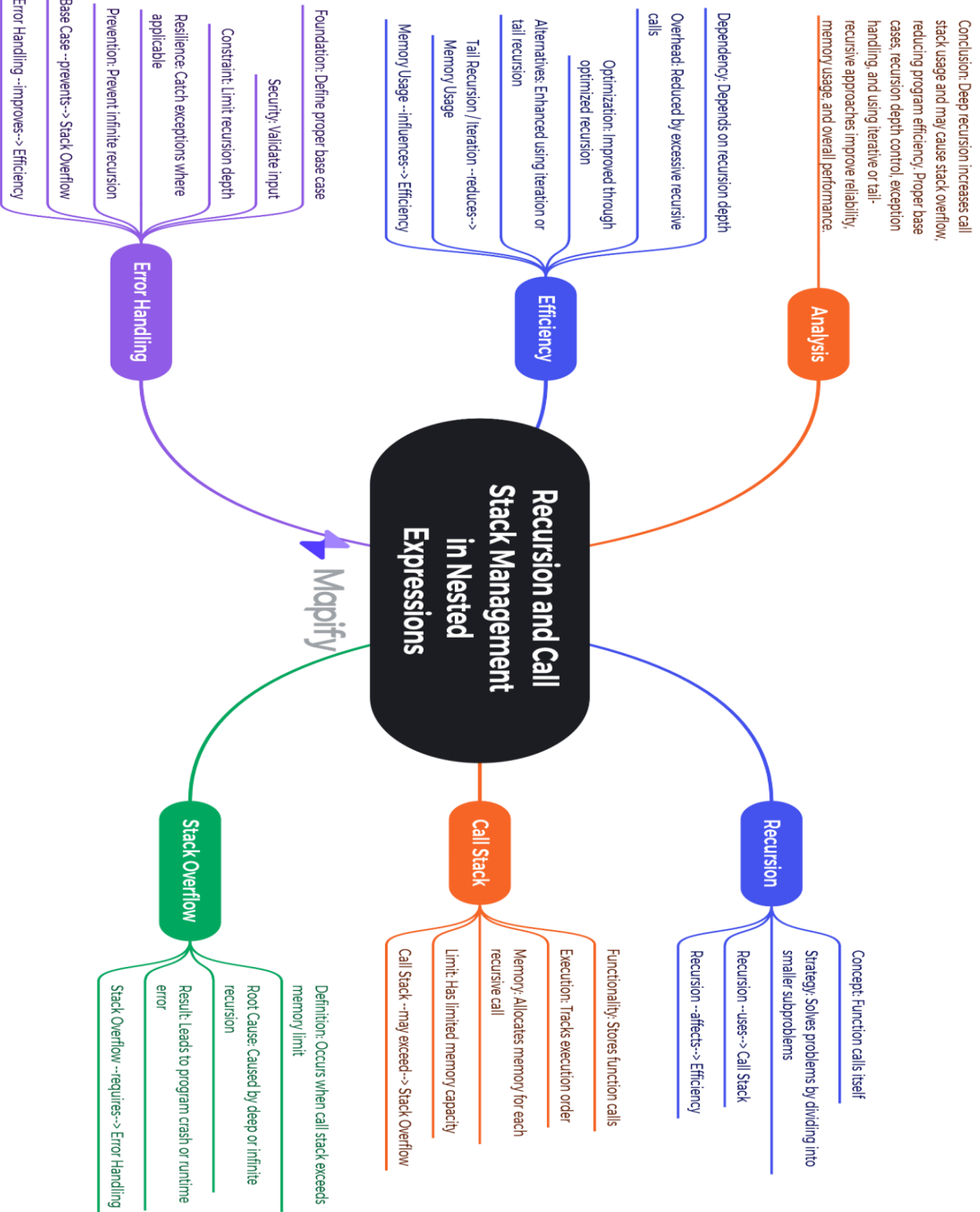
Weather Monitoring System — Data Structures Concept Map



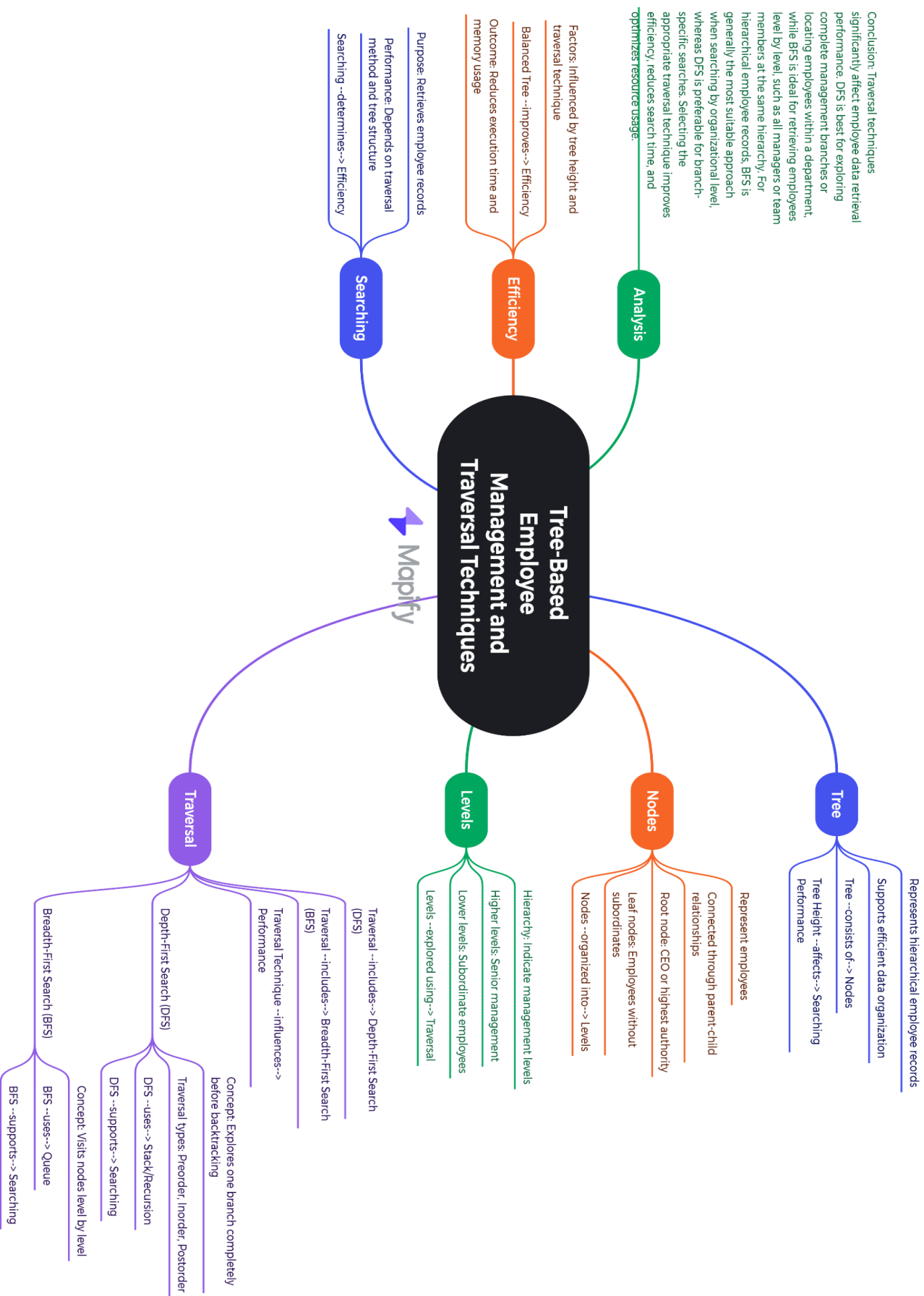
3.

Search Algorithms





4.



5.